

DEFINITIONS!

ADVANCED TOPICS IN ~~CYBERSECURITY~~ CRYPTOGRAPHY (7CCSMATC)

Martin R. Albrecht

OUTLINE

Introduction

Example 1: “What does it mean to generate random-looking bits?”

Example 2: “What does it mean for a block cipher to be secure?”

Example 3: “What does it mean for an encryption scheme to be secure?”

Exercise

INTRODUCTION

MAIN REFERENCE

Phillip Rogaway. **On the Role of Definitions in and Beyond Cryptography**. In: *Annual Asian Computing Science Conference*. Springer. 2004, pp. 13–32



- One of the most influential cryptographers in the world
- Many cryptographic solutions rely on definitions, models and constructions by him
- Fellow of the IACR
- Winner of the Levchin prize
- NSF CAREER Award in 1996^a

^aWhich the NSA had attempted to prevent by influencing the NSF.

MOTIVATION



kennyog @kennyog · Jan 9



We disclosed the vulnerabilities to [@ThreemaApp](#) back in October. Congrats to the team there on getting everything fixed quickly.



1



1



28



3,877



kennyog
@kennyog



Lessons learned 1: using a good crypto library is not enough – don't roll your own protocol.

MOTIVATION

kennvog @kennvog · Jan 9



Josh Baron

@JoshuaWBaron

Cryptography is hard: Russian malware edition

JOINT CYBERSECURITY ADVISORY

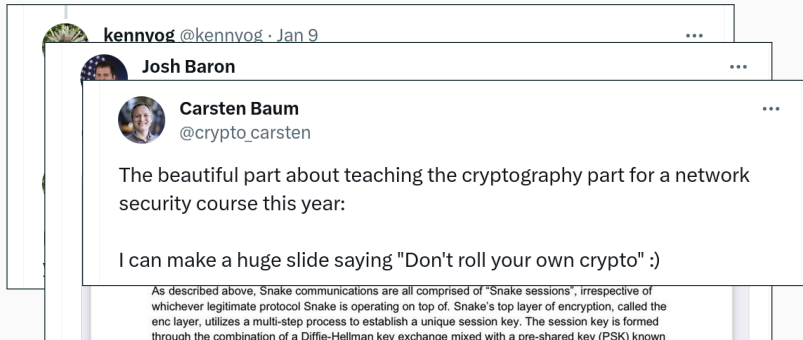
TLP: CLEAR

International Partnership

Snake "enc" Layer

As described above, Snake communications are all comprised of "Snake sessions", irrespective of whichever legitimate protocol Snake is operating on top of. Snake's top layer of encryption, called the enc layer, utilizes a multi-step process to establish a unique session key. The session key is formed through the combination of a Diffie-Hellman key exchange mixed with a pre-shared key (PSK) known

MOTIVATION



MOTIVATION

kennvog @kennvog · Jan 9

Josh Baron

Carsten Baum



Deirdre Connolly¹

@durumcrustulum

"Don't roll your own crypto^H^H^H^H^Hprotocol!"

securitycryptographywhatever.com/2021/07/31/the...

[#RealWorldCrypto](#)



securitycryptographywhatever.com

The Great

Special guest Filippo Valsorda joins us to debate with Thomas on whether one should or should not “roll your o...

MOTIVATION



“Crypto is hard!”



MOTIVATION



“Crypto is hard!”

Of course, cryptography is hard, so is any other science.

MOTIVATION

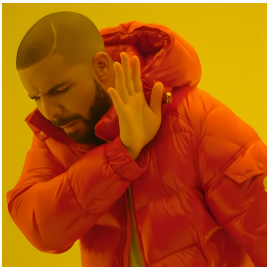


“Crypto is hard!”



Modern cryptography gives us the tools to reason about cryptographic protocols to rule out weaknesses.

MOTIVATION



“Crypto is hard!”



“Cryptography needs security models and proofs!”

You should neither design nor implement cryptographic primitives or protocols.¹

¹**You** should also neither design nor build bridges etc.

“PROVABLE SECURITY WOULD DRAMATICALLY CHANGE THE CHARACTER OF MY FIELD.” I

Definitions in cryptography emerged rather suddenly, in 1982, with the work of Shafi Goldwasser and Silvio Micali [GM84]. Before then, cryptography was all about schemes and attacks, and there was no way to gain confidence in a scheme beyond that which was had when smart people failed to find an attack.

“PROVABLE SECURITY WOULD DRAMATICALLY CHANGE THE CHARACTER OF MY FIELD.” II

What Goldwasser and Micali did was, first of all, to **define** cryptography's classical goal, message privacy (the goal of an **encryption scheme**).

The [GM84] definition was strong and satisfying, and they proved it equivalent to very different-looking alternatives.

Next they gave a **protocol** for encryption, and **proved** their protocol satisfied their definition, given a complexity-theoretic assumption.

The proof took the form of a **reduction**: if the encryption protocol didn't meet the security definition, some other protocol wouldn't satisfy its security definition.

The **definition-protocol-proof** approach has come to be called **provable security**.

Shafi Goldwasser and Silvio Micali.
Probabilistic Encryption. In:
*Journal of Computer and System
Sciences* 28.2 (1984), pp. 270–299^a

^aThis work earned the Turing Award in 2012: “For transformative work that laid the complexity-theoretic foundations for the science of cryptography and in the process pioneered new methods for efficient verification of mathematical proofs in complexity theory”



EXAMPLE 1: “WHAT DOES IT MEAN TO
GENERATE RANDOM-LOOKING BITS?”

IMPOSSIBILITY?

- We aim to produce pseudorandom bits that are so much like the genuine article that no feasible program $\mathcal{D}$ can tell the difference.
- This might sound impossible.
- If we generate any fixed string of pseudorandom bits y , we have no chance:
 - For any y there is a simple algorithm $\mathcal{D}$ that does a good job at telling if it is given y or random bits.
 - **What is that program?**

```
int get_random_number() {  
    return 4; // chosen by fair dice roll.  
             // guaranteed to be random.  
}
```

Credit: <https://xkcd.com/221/>

Take away

Our method of making pseudorandom bits must itself be randomised.

We therefore focus on an object, a **pseudorandom generator (PRG)**, that stretches a random **seed**, s , into a longer, pseudorandom string. Formally:

Definition (PRG Syntax)

A PRG is a function $G : \{0, 1\}^n \rightarrow \{0, 1\}^\ell$ where $\ell > n \geq 1$ are constants associated to G .

We therefore focus on an object, a **pseudorandom generator (PRG)**, that stretches a random **seed**, s , into a longer, pseudorandom string. Formally:

Definition (PRG Syntax)

A PRG is a function $G : \{0, 1\}^n \rightarrow \{0, 1\}^\ell$ where $\ell > n \geq 1$ are constants associated to G .

We are not done

This is just syntax: $G(s) = 0^\ell$ satisfies the API.

- We play a game between a challenger and a distinguisher $\mathcal{D}$ or **adversary** (often written $\mathcal{A}$).
- A distinguisher is just an algorithm, possibly a probabilistic one, equipped with a way to interact with its environment.
- We equip $\mathcal{D}$ with an **oracle** that has a “button” that $\mathcal{D}$ can push. Each time $\mathcal{D}$ pushes the button it gets a string of length ℓ .
- The distinguisher outputs a bit $b \in \{0, 1\}$ encoding its answer.

We play a game:

Game₀ every time the distinguisher $\mathcal{D}$ makes an oracle query, choose a random string $y \in \{0, 1\}^\ell$ and return it.

Game₁ every time the distinguisher $\mathcal{D}$ makes an oracle query, choose a random $x \in \{0, 1\}^n$ and give out $y = G(x)$.

Game ₀	G()
1: $b \leftarrow \mathcal{D}^G$	1: $y \leftarrow \$ \{0, 1\}^\ell$
2: return b	2: $x \leftarrow \$ \{0, 1\}^n$ // Game ₁
Game ₁	3: $y \leftarrow G(x)$ // Game ₁
1: $b \leftarrow \mathcal{D}^G$	4: return y
2: return b	

Figure 1: PRG Security Games.

- The distinguisher wants to decide if it is playing Game_0 or Game_1 .
 - So it outputs a bit b and then halts, where b matches the game it thinks it is playing.
- The advantage of $\mathcal{D}$ in attacking G is defined as

$$\text{Adv}_G^{\text{prg}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]|.$$

- We write $\Pr[\mathcal{A}^{\text{Game}}]$ for the probability of $\mathcal{A}$ outputting one in the game Game .
 - We write $\Pr[\text{Game}^{\mathcal{A}}]$ for the probability of Game outputting one with adversary $\mathcal{A}$.
- For our bad generator $G(x) = 0^\ell$ there is an easy distinguisher: If the output is all zero, return 0, otherwise return 1.

CONCRETE SECURITY

We say a PRG is secure if no “efficient” $\mathcal{D}$ exists:

$$\forall \mathcal{D} \in t \text{ steps} : \text{Adv}_G^{\text{prg}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon.$$

For example $(n, t, \varepsilon) = (128, 2^{64}, 2^{-64})$, s.t. $t/\varepsilon = 2^{128}$.

CONCRETE SECURITY

We say a PRG is secure if no “efficient” $\mathcal{D}$ exists:

$$\forall \mathcal{D} \in t \text{ steps} : \text{Adv}_G^{\text{prg}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon.$$

For example $(n, t, \varepsilon) = (128, 2^{64}, 2^{-64})$, s.t. $t/\varepsilon = 2^{128}$.

- 2GHz CPU: $\approx 2^{31} = 2 \cdot 2^{3 \cdot 10}$ ops per second
- $3600 \cdot 24 \cdot 365 \cdot 100 \approx 2^{11} \cdot 2^5 \cdot 2^8 \cdot 2^7 \approx 2^{31}$ seconds in 100 years
- 190 billion $\approx 2^{38}$ ARM chips by 2020.

All ARM cores **ever** clocked at 2GHz for 100 years gives you only 2^{100} ops.^a

^aAll Bitcoin miners together perform about 2^{68} hashes per second.

CONCRETE SECURITY

We say a PRG is secure if no “efficient” $\mathcal{D}$ exists:

$$\forall \mathcal{D} \in t \text{ steps} : \text{Adv}_G^{\text{prg}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon.$$

For example $(n, t, \varepsilon) = (128, 2^{64}, 2^{-64})$, s.t. $t/\varepsilon = 2^{128}$.

- 2GHz CPU: $\approx 2^{31} = 2 \cdot 2^{3 \cdot 10}$ ops per second
- $3600 \cdot 24 \cdot 365 \cdot 100 \approx 2^{11} \cdot 2^5 \cdot 2^8 \cdot 2^7 \approx 2^{31}$ seconds in 100 years
- 190 billion $\approx 2^{38}$ ARM chips by 2020.
- $(t, \varepsilon) = (2^{128}, 1)$?

All ARM cores **ever** clocked at 2GHz for 100 years gives you only 2^{100} ops.^a

^aAll Bitcoin miners together perform about 2^{68} hashes per second.

CONCRETE SECURITY

We say a PRG is secure if no “efficient” $\mathcal{D}$ exists:

$$\forall \mathcal{D} \in t \text{ steps} : \text{Adv}_G^{\text{prg}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon.$$

For example $(n, t, \varepsilon) = (128, 2^{64}, 2^{-64})$, s.t. $t/\varepsilon = 2^{128}$.

- 2GHz CPU: $\approx 2^{31} = 2 \cdot 2^{3 \cdot 10}$ ops per second
- $3600 \cdot 24 \cdot 365 \cdot 100 \approx 2^{11} \cdot 2^5 \cdot 2^8 \cdot 2^7 \approx 2^{31}$ seconds in 100 years
- 190 billion $\approx 2^{38}$ ARM chips by 2020.

• $(t, \varepsilon) = (2^{128}, 1)$?

- There might be a fast algorithm that succeeds with probability $1/3$.
- Often can run our $(t, \varepsilon) = (2^{64}, 2^{-64})$ attacker 2^{64} times to get $\approx (2^{128}, 1)$.

All ARM cores **ever** clocked at 2GHz for 100 years gives you only 2^{100} ops.^a

^aAll Bitcoin miners together perform about 2^{68} hashes per second.

CONCRETE SECURITY

We say a PRG is secure if no “efficient” $\mathcal{D}$ exists:

$$\forall \mathcal{D} \in t \text{ steps} : \text{Adv}_G^{\text{prg}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon.$$

For example $(n, t, \varepsilon) = (128, 2^{64}, 2^{-64})$, s.t. $t/\varepsilon = 2^{128}$.

- 2GHz CPU: $\approx 2^{31} = 2 \cdot 2^{3 \cdot 10}$ ops per second
- $3600 \cdot 24 \cdot 365 \cdot 100 \approx 2^{11} \cdot 2^5 \cdot 2^8 \cdot 2^7 \approx 2^{31}$ seconds in 100 years
- 190 billion $\approx 2^{38}$ ARM chips by 2020.

All ARM cores **ever** clocked at 2GHz for 100 years gives you only 2^{100} ops.^a

- $(t, \varepsilon) = (2^{128}, 1)$?
 - There might be a fast algorithm that succeeds with probability $1/3$.
 - Often can run our $(t, \varepsilon) = (2^{64}, 2^{-64})$ attacker 2^{64} times to get $\approx (2^{128}, 1)$.
- $(t, \varepsilon) = (1, 2^{-128})$?

^aAll Bitcoin miners together perform about 2^{68} hashes per second.

CONCRETE SECURITY

We say a PRG is secure if no “efficient” $\mathcal{D}$ exists:

$$\forall \mathcal{D} \in t \text{ steps} : \text{Adv}_G^{\text{prg}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon.$$

For example $(n, t, \varepsilon) = (128, 2^{64}, 2^{-64})$, s.t. $t/\varepsilon = 2^{128}$.

- 2GHz CPU: $\approx 2^{31} = 2 \cdot 2^{3 \cdot 10}$ ops per second
- $3600 \cdot 24 \cdot 365 \cdot 100 \approx 2^{11} \cdot 2^5 \cdot 2^8 \cdot 2^7 \approx 2^{31}$ seconds in 100 years
- 190 billion $\approx 2^{38}$ ARM chips by 2020.

All ARM cores **ever** clocked at 2GHz for 100 years gives you only 2^{100} ops.^a

^aAll Bitcoin miners together perform about 2^{68} hashes per second.

- $(t, \varepsilon) = (2^{128}, 1)$?
 - There might be a fast algorithm that succeeds with probability $1/3$.
 - Often can run our $(t, \varepsilon) = (2^{64}, 2^{-64})$ attacker 2^{64} times to get $\approx (2^{128}, 1)$.
- $(t, \varepsilon) = (1, 2^{-128})$?
 - There might be an algorithm that runs in two steps but succeeds with very high probability.

We say a PRG is secure if no “efficient” $\mathcal{D}$ exists:

$$\forall \mathcal{D} \in \text{PPT}: \text{Adv}_G^{\text{prg}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < 1/\text{poly}(n).$$

poly(n) any polynomial in n , e.g. n^2, n^{1000}, n^c where c is a constant

PPT probabilistic polynomial-time, an algorithm running in any polynomial time that may make random choices.

ON $< 1/\text{poly}(n)$ AKA $\in \text{negl}(n)$

- We insist on advantage $< 1/\text{poly}(n)$, which we write as $\in \text{negl}(n)$, because that rules out a polynomial-time adversary amplifying the success probability.
- Let m be the number of attempts by $\mathcal{D}$ and let win_i be the event that it succeeded in attempt i .
- Let p be an upper bound on the success probability of $\mathcal{D}$. Then, overall we have

$$\Pr[\text{win}] = \Pr\left[\bigvee_{i=1}^m \text{win}_i\right] \leq \sum_{i=1}^m \Pr[\text{win}_i] \leq m \cdot p,$$

where the second last bound is from the “union bound”.

- If $p \in \text{negl}(n)$ then $\text{poly}(n) \cdot \text{negl}(n) \in \text{negl}(n)$ because $n^{\omega(1)-c} \in n^{\omega(1)}$ for any constant c .²

²You can think of $\log(n)$ or $\log \log(n)$ as examples of functions $\in \omega(1)$.

PUTTING IT ALL TOGETHER

Definition (PRG)

A PRG is a function $G : \{0, 1\}^n \rightarrow \{0, 1\}^\ell$ where $\ell > n \geq 1$ are constants associated to G . We say G is (t, ε) -secure if

$$\forall \mathcal{D} \in t \text{ steps: } \text{Adv}_G^{\text{prg}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon$$

for Game_0 and Game_1 defined in Fig. 1.

We say G is secure if $t = \text{poly}(n)$ and $\varepsilon = 1/\text{poly}(n)$.

EXAMPLE 2: “WHAT DOES IT MEAN
FOR A BLOCK CIPHER TO BE SECURE?”

Definition (Block Cipher)

A blockcipher is a function $E : \mathcal{K} \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ where

- $\mathcal{K}$ is a finite, nonempty set (the key space),
- $n \geq 1$ is a number (the blocksize) and
- $E_k(\cdot)$ is a permutation (on $\{0, 1\}^n$) for each $k \in \mathcal{K}$.

When $y := E_k(x)$ we call x the input block and y the output block.

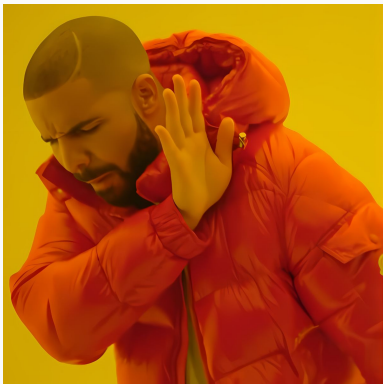
We may also define $x := E_k^{-1}(y)$ s.t. $y = E_k(x)$. We may write $D_k(\cdot) := E_k^{-1}(\cdot)$.

Permutation For every $y \in \{0, 1\}^n$ and every k there exists **one** x s.t. $y = E_k(x)$.

Example AES is **the** block cipher.

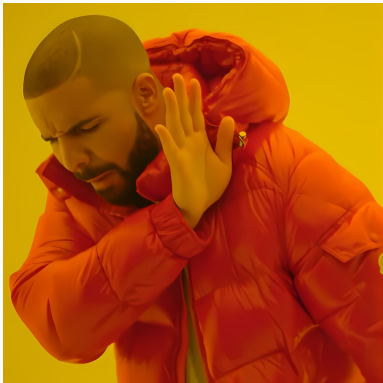
What do you think should be the definition of block cipher security?

SECURITY: WRONG DEFINITIONS



“Adversary looking at the output does not learn the key.”

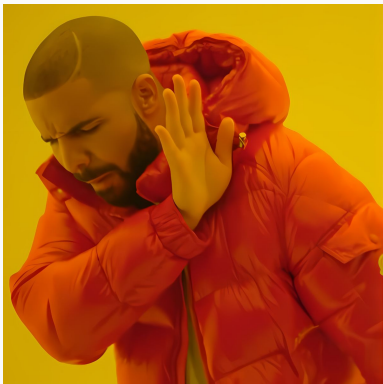
SECURITY: WRONG DEFINITIONS



“Adversary looking at the output does not learn the key.”

```
def E(k, x):  
    return x
```

SECURITY: WRONG DEFINITIONS

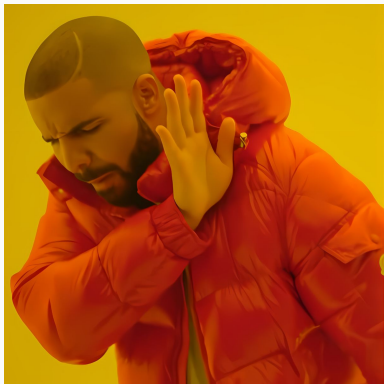


“Adversary looking at the output does not learn the key.”

```
def E(k, x):  
    return x
```

“Adversary looking at the output does not learn the input.”

SECURITY: WRONG DEFINITIONS



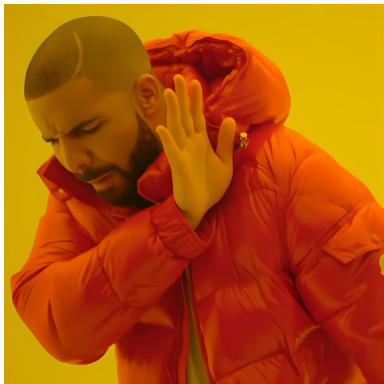
“Adversary looking at the output does not learn the key.”

```
def E(k, x):  
    return x
```

“Adversary looking at the output does not learn the input.”

```
def E(k, x):  
    y = (AES128(k, x[0]), x[1])  
    return y
```

SECURITY: WRONG DEFINITIONS



“Adversary looking at the output does not learn the key.”

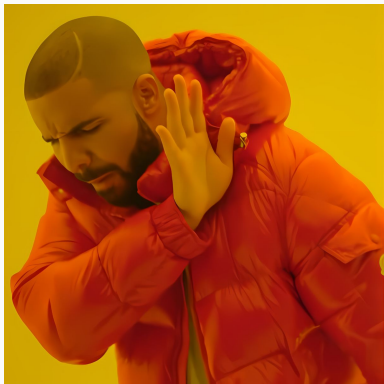
```
def E(k, x):  
    return x
```

“Adversary looking at the output does not learn the input.”

```
def E(k, x):  
    y = (AES128(k, x[0]), x[1])  
    return y
```

“Adversary looking at the output learns nothing about the input.”

SECURITY: WRONG DEFINITIONS



“Adversary looking at the output does not learn the key.”

```
def E(k, x):  
    return x
```

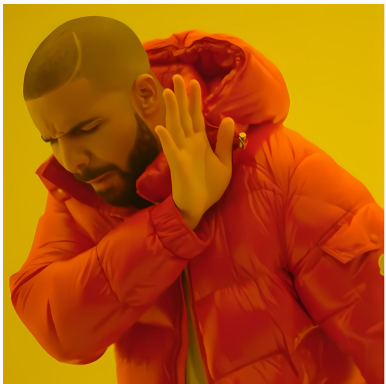
“Adversary looking at the output does not learn the input.”

```
def E(k, x):  
    y = (AES128(k, x[0]), x[1])  
    return y
```

“Adversary looking at the output learns nothing about the input.”

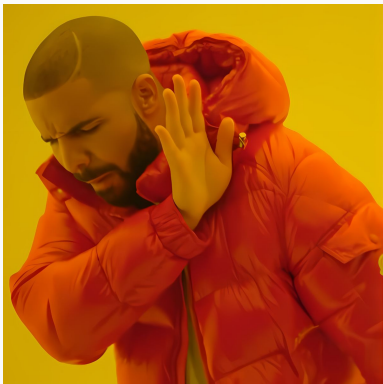
$$\Rightarrow |x| = n$$

SECURITY: WRONG DEFINITIONS



“Adversary looking at the output learns nothing about the input except its length.”

SECURITY: WRONG DEFINITIONS



“Adversary looking at the output learns nothing about the input except its length.”

1. Bob asks “Do you like pizza?” in the clear
2. Alice responds with $E_k(\text{YES!DEFINITELY!!})$
3. Bob asks “Do you want to commit all the crimes?” in the clear?
4. Alice responds $E_k(\text{YES!DEFINITELY!!})$

Adversary knows Alice responded with the same answer to the two questions.

Definitions can be wrong.

- Block ciphers are not encryption schemes.
- The **Advanced Encryption Standard** (AES) is **not** an encryption scheme.
- You know this result as “Don’t use ECB mode”.

SECURITY: PSEUDORANDOM PERMUTATION (PRP)

Game₀ a random permutation $\pi : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is chosen and when $\mathcal{D}$ asks its oracle a question x , we return $\pi(x)$.

Game ₀	P(x)
1: // n -bit perm.	1: $y \leftarrow \pi(x)$ // Game ₀
2: $\pi \leftarrow \mathcal{P}^n$	2: $y \leftarrow E_k(x)$ // Game ₁
3: return $\mathcal{D}^P$	3: return y
Game ₁	
1: $k \leftarrow \mathcal{K}$	
2: return $\mathcal{D}^P$	

Game₁ a random key $k \in \mathcal{K}$ is chosen and when $\mathcal{D}$ asks its oracle a question x we return $y = E_k(x)$.

At the end the distinguisher needs to output its guess for what world we live in:

$$\text{Adv}_E^{\text{prp}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]|.$$

SECURITY: PRP (NOPE!)

As it stands, we could not play this game. Why?

SECURITY: PRP (FIXED)

Game ₀	P(x)
1: $\pi \leftarrow \emptyset$	1: if $x \in \pi.\text{keys}$ then
2: return $\mathcal{D}^P$	2: $y \leftarrow \pi[x]$
Game ₁	3: else
1: $\pi \leftarrow \emptyset; k \leftarrow \mathcal{K}$	4: $y \leftarrow \{0, 1\}^n \setminus \pi.\text{values}$
2: return $\mathcal{D}^P$	5: $\pi[x] \leftarrow y$
	6: $y \leftarrow E_k(x)$ //Game ₁
	7: return y

The adversary will make at most polynomially many queries so the size of the table π will be polynomially sized.

Figure 2: PRP Security Games.

$E_k(\cdot)$, BUT WHAT ABOUT $E_k^{-1}(\cdot)$

We covered that the adversary might be able to call $E_k(\cdot)$ but not $E_k^{-1}(\cdot)$.

SECURITY: STRONG PSEUDORANDOM PERMUTATION (SPRP)

Game ₀	P(x)	P ⁻¹ (y)
1: $\pi \leftarrow \emptyset$	1: if $x \in \pi.\text{keys}$ then	1: if $y \in \pi.\text{values}$ then
2: return $\mathcal{D}^{P, P^{-1}}$	2:	2: // Find the x s.t. $\pi[x] = y$
Game ₁	3: $y \leftarrow \pi[x]$	3: $x \leftarrow \pi^{-1}[y]$
1: $k \leftarrow \mathcal{K}$	4: else	4: else
2: return $\mathcal{D}^{P, P^{-1}}$	5: $y \leftarrow \{0, 1\}^n \setminus \pi.\text{values}$	5: $x \leftarrow \{0, 1\}^n \setminus \pi.\text{keys}$
	6: $\pi[x] \leftarrow y$	6: $\pi[x] \leftarrow y$
	7: $y \leftarrow E_k(x)$ //Game ₁	7: $x \leftarrow E_k^{-1}(y)$ //Game ₁
	8: return y	8: return x

Figure 3: SPRP Security Games.

$$\text{Adv}_E^{\text{sprp}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]|.$$

PUTTING IT ALL TOGETHER

Definition (PRP)

A PRP is a keyed permutation $E_k : \{0,1\}^n \rightarrow \{0,1\}^n$ for $k \leftarrow \mathcal{K}$.

We say E is (t, ε) -secure **PRP** if for Game_0 and Game_1 defined in Fig. 2 we have:

$$\forall \mathcal{D} \in t \text{ steps: } \text{Adv}_E^{\text{prp}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon .$$

We say E is (t, ε) -secure **SPRP** if for Game_0 and Game_1 defined in Fig. 3 we have:

$$\forall \mathcal{D} \in t \text{ steps: } \text{Adv}_E^{\text{sprp}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon .$$

We say E is (prp-,sprp-)secure if $t = \text{poly}(n)$ and $\varepsilon = 1/\text{poly}(n)$.

- Ideally, we want $t/\varepsilon \approx 2^n$
- No PRP can be $t = 2^n$, $\varepsilon = 1/\text{poly}(n)$ secure, if $|\mathcal{K}| < 2^n$:

PUTTING IT ALL TOGETHER

Definition (PRP)

A PRP is a keyed permutation $E_k : \{0,1\}^n \rightarrow \{0,1\}^n$ for $k \leftarrow \mathcal{K}$.

We say E is (t, ε) -secure **PRP** if for Game_0 and Game_1 defined in Fig. 2 we have:

$$\forall \mathcal{D} \in t \text{ steps: } \text{Adv}_E^{\text{prp}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon .$$

We say E is (t, ε) -secure **SPRP** if for Game_0 and Game_1 defined in Fig. 3 we have:

$$\forall \mathcal{D} \in t \text{ steps: } \text{Adv}_E^{\text{sprp}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon .$$

We say E is (prp-,sprp-)secure if $t = \text{poly}(n)$ and $\varepsilon = 1/\text{poly}(n)$.

- Ideally, we want $t/\varepsilon \approx 2^n$
- No PRP can be $t = 2^n$, $\varepsilon = 1/\text{poly}(n)$ secure, if $|\mathcal{K}| < 2^n$: Try all possible $k \in \mathcal{K}$; there are only $|\mathcal{K}|$ many, but there are $(2^n)! \gg 2^n$ permutations.

THIS COVERS ALL INTUITIVE, FEASIBLE DEFINITIONS DISCUSSED ABOVE

- “Adversary looking at the output does not learn the key.” this would allow distinguishing
- “Adversary looking at the output does not learn the input.” this would allow distinguishing
- “Adversary looking at the output learns nothing about the input except its length.” output is random (up to permutation), but calling it twice returns the same value twice.

The correct definition might be unintuitive.

EXAMPLE 3: “WHAT DOES IT MEAN
FOR AN ENCRYPTION SCHEME TO BE
SECURE?”

Definition (Authenticated Encryption)

An AE-scheme is a pair of algorithms (E, D) where

- $E : \mathcal{K} \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ is a randomised algorithm,
- $D : \mathcal{K} \times \{0, 1\}^* \rightarrow \{0, 1\}^* \cup \{\perp\}$ is a deterministic algorithm,
- set $\mathcal{K} \subseteq \{0, 1\}^*$ is nonempty and finite,
- $|E_k(m)| = |m| + \tau$ for some constant τ and
- $D_k(c) = m$ whenever $c = E_k(m)$.

Algorithms E and D are called the encryption algorithm and the decryption algorithm, and strings k , m and c are called the key, plaintext, and ciphertext.

What do you think encryption should provide?

“Indistinguishability under Chosen Plaintext Attacks”

Game ₀	E(m ₀ , m ₁)
1: $k \leftarrow \mathcal{K}$	1: if $ m_0 \neq m_1 $ then
2: return $\mathcal{D}^E$	2: return $\perp$
Game ₁	3: $c \leftarrow E_k(m_0)$
1: $k \leftarrow \mathcal{K}$	4: $c \leftarrow E_k(m_1)$ //Game ₁
2: return $\mathcal{D}^E$	5: return c

$$\text{Adv}_E^{\text{ind-cpa}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]|$$

SECURITY: PRIVACY (IND-CPA) NEATER

IND-CPA	$E(m_0, m_1)$
1: $k \leftarrow \$ \mathcal{K}$	1: if $ m_0 \neq m_1 $ then
2: $b \leftarrow \$ \{0, 1\}$	2: return $\perp$
3: return $\mathcal{D}^E$	3: $c \leftarrow \$ E_k(m_b)$
	4: return c

$$\text{Adv}_E^{\text{ind-cpa}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{IND-CPA}} = 1 \mid b = 1] - \Pr[\mathcal{D}^{\text{IND-CPA}} = 1 \mid b = 0]|$$

SECURITY: PRIVACY (IND-CPA) NEATEST

IND-CPA	$E(m_0, m_1)$
1: $k \leftarrow \mathcal{K}$	1: if $ m_0 \neq m_1 $ then
2: $b \leftarrow \{0, 1\}$	2: return $\perp$
3: $b' \leftarrow \mathcal{D}^E$	3: $c \leftarrow E_k(m_b)$
4: return $b = b'$	4: return c

Figure 4: IND-CPA Security Game.

$$\text{Adv}_E^{\text{ind-cpa}}(\mathcal{D}) = \left| \Pr[\text{IND-CPA}^{\mathcal{D}} = 1] - 1/2 \right|$$

This form is exactly half of $|\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]|$, the form used above and for the PRG and PRP; we use the two forms interchangeably.

WHAT DOES IND-CPA GET YOU?

```
from Crypto.Cipher import AES
from io import StringIO
import gzip

cipher = AES.new(k, AES.MODE_GCM)

def compress(m):
    out = StringIO()
    with gzip.GzipFile(fileobj=out, mode="w") as f:
        f.write(m)
    return out.getvalue()

def encrypt(k, m): # IND-CPA secure
    c, t = cipher.encrypt_and_digest(m)
    return c, t
```

```
encrypt2 = lambda k, m: encrypt(k, compress(m))
```



WIKIPEDIA
The Free Encyclopedia



CRIME

6 languages

Article [Talk](#)

Tools

From Wikipedia, the free encyclopedia

For criminal activity, see [Crime](#). For other uses, see [Crime \(disambiguation\)](#).

CRIME (Compression Ratio Info-leak Made Easy) is a [security vulnerability](#) in [HTTPS](#) and [SPDY](#) protocols that utilize compression, which can leak the content of secret [web cookies](#).^[1] When used to recover the content of secret [authentication cookies](#), it allows an attacker to perform [session hijacking](#) on an authenticated web session, allowing the launching of further attacks. CRIME was assigned [CVE-2012-4929](#).^[2]

Details [\[edit \]](#)

The vulnerability exploited is a combination of [chosen plaintext attack](#) and inadvertent [information leakage](#) through data compression, similar to that described in 2002 by the cryptographer [John Kelsey](#).^[3] It relies on the attacker being able to observe the size of the [ciphertext](#) sent by the [browser](#) while at the same time inducing the browser to make multiple carefully crafted web connections to the target site. The attacker then

SECURITY: INTEGRITY (INT-CTXT)

“Integrity of Ciphertexts”

INT-CTXT	$E(m)$
1: $\mathcal{C} \leftarrow \emptyset; k \leftarrow \$ \mathcal{K}$	1: $c \leftarrow \$ E_k(m)$
2: $c \leftarrow \mathcal{A}^E$	2: $\mathcal{C} \leftarrow \mathcal{C} \cup \{c\}$
3: $m \leftarrow D_k(c)$	3: return c
4: if $c \notin \mathcal{C}$ and $m \neq \perp$ then return 1	
5: else return 0	

Figure 5: INT-CTXT Security Game.

$$\text{Adv}_E^{\text{int-ctxt}}(\mathcal{D}) = \Pr[\text{INT-CTXT}^{\mathcal{D}} = 1].$$

SECURITY: INTEGRITY (INT-CTXT) NEATER

INT-CTXT	$E(m)$	$D(c)$
1: $\mathcal{C} \leftarrow \emptyset$	1: $c \leftarrow \$ E_k(m)$	1: if $b = 1$ then
2: $k \leftarrow \$ \mathcal{K}$	2: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(m, c)\}$	2: return $D_k(c)$
3: $b \leftarrow \$ \{0, 1\}$	3: return c	3: else if $\exists (m, c) \in \mathcal{C}$ then
4: $b' \leftarrow \mathcal{A}^{E,D}$		4: return m
5: return $b = b'$		5: else return $\perp$

$$\text{Adv}_{E,D}^{\text{int-ctxt}}(\mathcal{D}) = \left| \Pr[\text{INT-CTXT}^{\mathcal{D}} = 1] - 1/2 \right|.$$

DIDN'T WE FORGET SOMETHING?

- We defined PRP and SPRP security for block ciphers.

DIDN'T WE FORGET SOMETHING?

- We defined PRP and SPRP security for block ciphers.
- Our IND-CPA encryption scheme only provides an encryption oracle.
- Should we not also provide a decryption oracle?

“Indistinguishability under Chosen Ciphertext Attacks”

IND-CCA	$E(m_0, m_1)$	$D(c)$
1: $\mathcal{C} \leftarrow \emptyset$	1: if $ m_0 \neq m_1 $ then	1: if $c \in \mathcal{C}$ then
2: $k \leftarrow \$ \mathcal{K}$	2: return $\perp$	2: return $\perp$
3: $b \leftarrow \$ \{0, 1\}$	3: $c \leftarrow \$ E_k(m_b)$	3: return $D_k(c)$
4: $b' \leftarrow \mathcal{D}^{E,D}$	4: if $m_0 \neq m_1$ then $\mathcal{C} \leftarrow \mathcal{C} \cup \{c\}$	
5: return $b = b'$	5: return c	

Figure 6: IND-CCA Security Game.

$$\text{Adv}_{E,D}^{\text{ind-cca}}(\mathcal{D}) = \left| \Pr[\text{IND-CCA}^{\mathcal{D}} = 1] - 1/2 \right|.$$

IND-CPA + INT-CTXT $\Rightarrow$ IND-CCA

Claim: If you have IND-CPA security and INT-CTXT security then you have IND-CCA security.

Argument Sketch: Simulate the decryption oracle!

Game ₀	$E(m_0, m_1)$	$D(c)$
1: $T \leftarrow \emptyset$	1: if $ m_0 \neq m_1 $ then	1: if $c \in T.\text{keys}$ then
2: $k \leftarrow \$ \mathcal{K}$	2: return $\perp$	2: return $T[c]$
3: $b \leftarrow \$ \{0, 1\}$	3: $c \leftarrow \$ E_k(m_b)$	3: return $\perp$
4: $b' \leftarrow \mathcal{D}^{E,D}$	4: if $m_0 = m_1$ then $T[c] \leftarrow m_b$	
5: return $b = b'$	5: return c	

Either IND-CCA adversary $\mathcal{D}$ submits some $c \notin T.\text{keys}$ to D that decrypts correctly (breaking INT-CTXT) or it did not (breaking IND-CPA).

$\neg \text{INT-CTXT} \not\Rightarrow \neg \text{IND-CCA I}$

Let $S = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be a given IND-CCA secure symmetric encryption scheme. We define a scheme S_1 that is IND-CCA secure but is not INT-CTXT secure.³

Idea:

- A certain known string will be viewed by Dec_1 as valid and decrypted to certain known messages, so that forgery is easy.
- But these “ciphertexts” will never be produced by the encryption algorithm so privacy will not be affected.

³Mihir Bellare and Chanathip Namprempre. **Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm**. In: *Journal of Cryptology* 21.4 (Oct. 2008), pp. 469–491. DOI: 10.1007/s00145-008-9026-x.

The new scheme $S_1 = (\text{KeyGen}, \text{Enc}_1, \text{Dec}_1)$ has the same key generation algorithm as the old scheme and the following modified encryption and decryption algorithms:

 $\text{Enc}_1(k, m)$

1: $c' \leftarrow \text{Enc}(k, m)$

2: $c \leftarrow 0 \| c'$

3: **return** c

 $\text{Dec}_1(k, c)$

1: Parse c as $b \| c'$ where b is a bit

2: **if** $b = 0$ **then** $m \leftarrow \text{Dec}(k, c')$ **else** $m \leftarrow 0$

3: **return** m

“HAVING DEFINITIONS MAKES IT EASIER TO COME UP WITH ATTACKS.”

*[T]he U.S. National Security Agency (NSA) released, through NIST, its own scheme for efficient AE. The mechanism was called **dual counter mode**. I myself read the definition of the mode and broke it in less than a day, privately informing NIST. Donescu, Gligor, and Wagner likewise broke the scheme right away [DGW01]. What is it that we knew that the folks from the super-secret NSA did not? The answer, I believe, is **definitions**. If you understood the definition for AE, it was pretty obvious that the NSA scheme wouldn't work. In the absence of understanding a definition, you wouldn't see it. Definitions, not initially intended to help people find attacks, nonetheless do just that.*

Pompiliu Donescu, Virgil D Gligor, and David Wagner. **A note on NSA's Dual Counter Mode of encryption**. <https://people.eecs.berkeley.edu/~daw/papers/dcm-prelim.pdf>.
2001

EXERCISE

WRITE DOWN THE DEFINITION OF A PSEUDORANDOM FUNCTION (PRF)

WRITE DOWN THE DEFINITION OF A PSEUDORANDOM FUNCTION (PRF)

Definition (PRF)

A PRF is a keyed function $F_k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\ell$ where ℓ depends on λ , with $k \leftarrow \$ \mathcal{K}$. We say F_k is (t, ε) -secure **PRF** if for Game_0 and Game_1 defined below we have:

$$\forall \mathcal{D} \in t \text{ steps: } \text{Adv}_F^{\text{prf}}(\mathcal{D}) = |\Pr[\mathcal{D}^{\text{Game}_1} = 1] - \Pr[\mathcal{D}^{\text{Game}_0} = 1]| < \varepsilon$$

Game ₀	F(x)
1: $f \leftarrow \emptyset$	1: if $x \notin f.\text{keys}$ then $f[x] \leftarrow \$ \{0, 1\}^\ell$
2: return $\mathcal{D}^f$	2: $y \leftarrow f[x]$
Game ₁	3: $y \leftarrow F_k(x)$ //Game ₁
1: $f \leftarrow \emptyset; k \leftarrow \$ \mathcal{K}$	4: return y
2: return $\mathcal{D}^f$	

SAY “NO” TO EMPTY ABSTRACTIONS



“THIS CRYPTOGRAPHIC SCHEME IS
SECURE.”



“THIS CRYPTOGRAPHIC SCHEME
ACHIEVES IND-CCA SECURITY,
WHICH MEANS...”

- [BN08] Mihir Bellare and Chanathip Namprempre. **Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm**. In: *Journal of Cryptology* 21.4 (Oct. 2008), pp. 469–491. DOI: 10.1007/s00145-008-9026-x.
- [DGW01] Pompiliu Donescu, Virgil D Gligor, and David Wagner. **A note on NSA's Dual Counter Mode of encryption**.
<https://people.eecs.berkeley.edu/~daw/papers/dcm-prelim.pdf>. 2001.
- [GM84] Shafi Goldwasser and Silvio Micali. **Probabilistic Encryption**. In: *Journal of Computer and System Sciences* 28.2 (1984), pp. 270–299.
- [Rog04] Phillip Rogaway. **On the Role of Definitions in and Beyond Cryptography**. In: *Annual Asian Computing Science Conference*. Springer. 2004, pp. 13–32.